

FATEC
Ipiranga

Análise e Desenvolvimento de Sistemas

Programação Estruturada e Modular



Modularização em C: Criação e Uso de Bibliotecas com #include

Professor: Veríssimo - carlos.pereira01@cps.sp.gov.br

09/03/2026

Sobre este documento

Este documento objetiva deixar registrado o conteúdo abordado em sala de aula pelo professor. Importante destacar que a Nota de Aula serve como guia ao professor, bem como serve aos alunos como um norte, quanto ao conteúdo desenvolvido em sala de aula.

Este documento não tem a pretensão de ser uma única fonte para estudo. Para tal, o aluno deverá assistir às aulas e fazer uso (consulta) à bibliografia recomendada na ementa da disciplina, e à bibliografia complementar, apontada pelo professor.

Objetivos da Aula

Ao final desta aula, você deverá ser capaz de:

- Compreender o conceito de modularização
- Separar código em arquivos `.h` e `.c`
- Criar bibliotecas próprias
- Utilizar corretamente a diretiva `include`
- Compilar programas com múltiplos arquivos

1. Motivação

Considere o seguinte código:

```
1 #include <stdio.h>
2
3 int soma(int a, int b) { return a + b; }
4 int sub(int a, int b) { return a - b; }
5
6 int main() {
7     printf("%d\n", soma(2,3));
8     printf("%d\n", sub(5,2));
9 }
```

Problemas:

- Código cresce rapidamente
- Dificuldade de manutenção
- Reutilização limitada

Solução: Modularização

2. Conceito de Modularização

Modularização consiste em dividir o programa em partes menores (módulos), cada uma com responsabilidade específica.

- **Arquivo .h (Header):** define a interface
- **Arquivo .c:** implementa as funções

3. Estrutura de uma Biblioteca em C

Arquivo de Cabeçalho: operacoes.h

```
1 #ifndef OPERACOES_H
2 #define OPERACOES_H
3
4 int soma(int a, int b);
5 int sub(int a, int b);
6
7 #endif
```

Arquivo de Implementação: operacoes.c

```
1 #include "operacoes.h"
2
3 int soma(int a, int b) {
4     return a + b;
5 }
6
7 int sub(int a, int b) {
8     return a - b;
9 }
```

Arquivo Principal: main.c

```
1 #include <stdio.h>
2 #include "operacoes.h"
3
4 int main() {
5     printf("%d\n", soma(2,3));
6     printf("%d\n", sub(5,2));
7 }
```

4. Diretiva #include

A diretiva `#include` é usada para incluir o conteúdo de um arquivo em outro.

- `#include <stdio.h>` → biblioteca padrão
- `#include "operacoes.h"` → biblioteca do usuário

Importante: o pré-processador copia o conteúdo do arquivo incluído.

5. Include Guards (Proteção contra Inclusão Múltipla)

O que são Include Guards?

Include guards são diretivas do pré-processador utilizadas para evitar que um mesmo arquivo `.h` seja incluído mais de uma vez durante a compilação.

Sem essa proteção, podem ocorrer erros como:

- Redefinição de funções

- Declarações duplicadas de estruturas
- Conflitos de variáveis

Problema sem Include Guard

Considere os seguintes arquivos:

A.h

```
1 int soma(int a, int b);
```

B.h

```
1 #include "A.h"
```

main.c

```
1 #include "A.h"  
2 #include "B.h"
```

Nesse caso, o arquivo **A.h** será incluído duas vezes:

- Diretamente em **main.c**
- Indiretamente através de **B.h**

Isso pode gerar erros de compilação.

Solução: Include Guard

Para evitar esse problema, utilizamos include guards:

```
1 #ifndef OPERACOES_H  
2 #define OPERACOES_H  
3  
4 int soma(int a, int b);  
5 int sub(int a, int b);  
6  
7 #endif
```

Como funciona

- `#ifndef OPERACOES_H`
Verifica se a constante ainda não foi definida
- `#define OPERACOES_H`
Define a constante
- O conteúdo do arquivo é incluído apenas se a constante ainda não existir
- `#endif`
Finaliza a condição

Funcionamento na prática

- Primeira inclusão:
 - A constante não existe
 - O conteúdo é incluído
 - A constante é definida
- Inclusões posteriores:
 - A constante já existe
 - O conteúdo é ignorado

Padrão recomendado

Sempre utilize o seguinte padrão em todos os arquivos `.h`:

```
1 #ifndef NOME_DO_ARQUIVO_H
2 #define NOME_DO_ARQUIVO_H
3
4 // declara es
5
6 #endif
```


Boas práticas para nomes

- Use nomes únicos e descritivos
- Evite nomes genéricos

Exemplo:

```
1 #ifndef PROJETO_OPERACOES_H
2 #define PROJETO_OPERACOES_H
```

Alternativa: pragma once

Existe também uma alternativa mais simples:

```
1 #pragma once
```

Porém:

- Não faz parte do padrão oficial da linguagem C
- Pode não ser suportado por todos os compiladores

Resumo

- Evita inclusão múltipla de arquivos
- Previne erros de compilação
- Deve ser usado em todo arquivo .h

6. Compilação

Para compilar múltiplos arquivos:

```
1 gcc main.c operacoes.c -o programa
```

Explicação:

- Cada arquivo `.c` é compilado
- O linker une tudo em um executável

6. Boas Práticas

- Utilize **include guards** em todos os arquivos `.h`
- Não implemente funções dentro do `.h`, apenas declare
- Use nomes de arquivos e funções significativos
- Separe responsabilidades (uma biblioteca para cada tema)
- Evite duplicação de código
- Documente funções com comentários
- Mantenha organização de pastas (ex: `src`, `include`)
- Compile com avisos ativados:

```
1 gcc -Wall -Wextra main.c operacoes.c -o  
   programa
```

8. Exemplo Completo Passo a Passo

Passo 1: Criar Estrutura de Pastas

```
1 projeto/  
2     include/  
3         operacoes.h  
4     src/  
5         operacoes.c  
6     main.c
```

Passo 2: Criar os Arquivos

include/operacoes.h

```
1 #ifndef OPERACOES_H
2 #define OPERACOES_H
3
4 int soma(int a, int b);
5 int sub(int a, int b);
6
7 #endif
```

src/operacoes.c

```
1 #include "operacoes.h"
2
3 int soma(int a, int b) {
4     return a + b;
5 }
6
7 int sub(int a, int b) {
8     return a - b;
9 }
```

main.c

```
1 #include <stdio.h>
2 #include "operacoes.h"
3
4 int main() {
5     printf("Soma: %d\n", soma(2,3));
6     printf("Subtracao: %d\n", sub(5,2));
7     return 0;
8 }
```

Passo 7: Compilar o Projeto

```
1 gcc main.c src/operacoes.c -Iinclude -o programa
```

Explicação:

- -Iinclude → indica onde estão os arquivos .h

- `-o programa` → nome do executável

Passo 4: Executar o Programa

No Linux/Mac:

```
1 ./programa
```

No Windows:

```
1 programa.exe
```

Saída Esperada

```
1 Soma: 5
2 Subtracao: 3
```

Conclusão

A modularização torna o código mais organizado, reutilizável e fácil de manter, sendo essencial em projetos maiores.